Ders 12 Çalışma Özeti

Ders 12 Çalışma Özeti

Genel Konular

- Dönem projesi hatırlatması.
- Kitabın 6. bölümü: sık öğe kümeleri (frequent itemsets).
- Pazar sepeti modeli (market basket model): öğeler (items) ve sepetler (baskets).
- Destek (support) eşiği ve sık öğe kümesi tanımı.
- Uygulamalar: süpermarket, intihal (plagiarism) tespiti, birlikte geçen kelimeler.
- Birliktelik kuralları (association rules) ve güven (confidence) eşiği.
- Sık öğe kümelerinden kural çıkarma.
- Verinin diskte saklanışı. Maliyet ölçüsü: disk geçişi (pass) sayısı.
- Çiftleri (pairs) bellekte sayma: üçgen matris (triangular matrix) ve üçlü tablo (triples) yaklaşımı.
- Saf (naive) algoritma, Apriori algoritması.
- Apriori'nin iyileştirmeleri: PCY, çok aşamalı (multistage) ve çok karmalı (multihash) algoritmalar.

Hocanın Özellikle Vurguladığı Kısımlar

- **Proje teslimi dönemin son haftasındaki ders saatinde.** O gün her grup projesini anlatacak ve demo yapacak. Hoca projeyle ilgili ve projenin tamamlanıp tamamlanmadığına dair sorular soracak.
- **Proje ağırlığı vizeyle aynı: %30.** Soruları e-postayla sorun, cevapsız soru bırakmayın.
- 6. bölümün slaytları ders sitesinde.
- **Birliktelik kuralı madenciliğinde zor kısım sık öğe kümelerini bulmaktır.** Sık kümeler bulunduktan sonra kural çıkarmak kolaydır.
- **Bu algoritmalarda maliyet CPU süresiyle değil, disk geçişi sayısıyla ölçülür.** Diske her erişim CPU'ya göre çok pahalıdır.
- **Ana bellek kritik kaynaktır.** Sayılacak her şey bellekte sayılmalıdır.
- Uygulamada en çok sık çiftler kullanılır. Bu yüzden derste yalnız çiftler işleniyor. Üçlüler ve dörtlüler kapsam dışı.
- **Apriori'nin temeli monotonluktur:** Bir küme sıksa tüm alt kümeleri de sıktır. Bir öğe sık değilse onu içeren hiçbir çift sık olamaz.
- PCY, multistage ve multihash, büyük veri kümelerinde sık öğe kümesi bulmak için kullanılan güncel algoritmalardır.

Kısa Tekrar Notları

- **Destek:** Bir öğe kümesini içeren sepet sayısı. Destek $\geq s$ ise küme sıktır.
- **Birliktelik kuralı:** $\{i_1, \ldots, i_k\} \rightarrow j$. Sol taraftaki öğeleri içeren sepet büyük olasılıkla j'yi de içerir.
- **Güven:** $conf(I \rightarrow j) = \frac{support(I \cup \{j\})}{support(I)}$.
- Kural madenciliği için iki eşik verilir: destek eşiği s ve güven eşiği c.
- **Saf algoritma:** Tüm çiftler bellekte tek geçişte sayılır. n öğe için $\binom{n}{2} = \frac{n(n-1)}{2}$ çift vardır. Bellek ihtiyacı yaklaşık $2n^2$ bayttır.
- **Üçgen matris:** Olası her çift için 4 bayt.
- **Üçlü tablo** $[i, j, sayı]$**:** Yalnız gerçekten görülen çiftler için 12 bayt.

- **Hangisi?** Görülen çift sayısı olası çiftlerin 1/3'ünden azsa üçlü tablo daha az bellek kullanır. Fazlaysa üçgen matris daha az kullanır.
- **Apriori:** 2 geçiş.
  - Geçiş 1: Tekil öğeleri say. Destek $\geq s$ olanlar sık öğelerdir.
  - Geçiş 2: Yalnız iki öğesi de sık olan çiftleri say.
- **PCY:** Geçiş 1'de boş kalan belleğe çiftlerin sayılarını tutan bir karma tablosu (hash table) koyar. Geçiş 2'de bir çift ancak şu iki koşulu sağlarsa sayılır:
  - İki öğesi de sık.
  - Sık bir kovaya (bucket) karmalanıyor.
- **Bit eşlem (bitmap):** 4 baytlık kova sayacı yerine 1 bit. Yer ihtiyacı 1/32'ye iner.
- **Multistage:** 3 geçiş. İkinci geçişte ikinci bir karma tablosu (farklı karma fonksiyonuyla) kullanır. Aday çift iki bit eşlemden de geçmelidir.
- **Multihash:** 2 geçiş. İlk geçişte boş belleği iki karma tablosuna böler. Aday çift iki bit eşlemden de geçmelidir.

Detaylı Açıklamalar

Pazar sepeti modeli

- Büyük bir öğe kümesi vardır, örneğin süpermarkette satılan ürünler.
- Büyük bir sepet kümesi vardır. Her sepet, kasaya getirilen ürünlerdir. Sepete işlem (transaction) de denir.
- Amaç, aynı sepette sık birlikte bulunan öğeleri bulmaktır.
- Örnek: Ekmek ve süt çoğu zaman birlikte satılıyorsa bu, pazarlama bölümü için önemli bir bilgidir.

Destek eşiği örneği

- Örnekte 5 farklı öğe (m, c, b, j, p) ve 8 sepet var. Destek eşiği $s = 3$.
- Sık tekil öğeler: m, c, b ve j.
- Sık çiftler: $\{m, b\}$ ve $\{b, c\}$ (ikisinin de desteği 4), ayrıca $\{c, j\}$.
- Sık kümeler tekli, ikili, üçlü vb. olabilir.

Uygulamalar

- **Süpermarket:** Problemin kökeni buradadır, bu yüzden "pazar sepeti analizi" denir.
  - Birlikte satılan iki ürün mağazanın iki ucuna konabilir. Müşteri mağazada dolaşır ve daha çok ürün alır.
  - Ya da iki ürüne birlikte indirim yapılıp yan yana konabilir.
- **İntihal tespiti:**
  - Öğeler = belgeler, sepetler = cümleler.
  - Aynı cümleleri sık paylaşan belgeler birbirine benzer. Bu belgeler intihal adayıdır.
- **Birlikte geçen kelimeler:**
  - Öğeler = kelimeler, sepetler = belgeler.
  - Birçok belgede birlikte geçen kelimeler bulunur.
- **Ölçek sorunu:** Walmart yaklaşık 100.000 ürün satar ve milyarlarca sepeti vardır. Web'de milyarlarca kelime ve sayfa vardır. Hem hesaplama hem bellek ihtiyacı tek bilgisayarı aşar.

Birliktelik kuralları ve güven

- Kural: $\{i_1, \ldots, i_k\} \to j$. Sol taraf ve j birlikte bir sık öğe kümesidir.
- **Güven:** Sol tarafı içeren sepetlerden kaçının j'yi de içerdiği. Yani $P(j \mid i_1, \ldots, i_k)$.
- Destek eşiği sık kümeleri bulmak için kullanılır. Güven eşiği kuralın geçerli olup olmadığına karar verir.
- **Örnek:** Güven eşiği %50. Kural: $\{m, b\} \to c$.
  - $\{m, b\}$ içeren sepet sayısı 4.
  - Bunlardan c'yi de içeren sepet sayısı 2.
  - Güven: $2/4 = \%50$. Kural eşiği sağlar.
- Hocanın tanımında bir kuralın desteği, sol taraftaki öğe kümesinin desteğidir.

Sık kümelerden kural çıkarma

- Önce destek eşiğiyle sık kümeler bulunur.
- Sonra her sık kümeden sırayla bir öğe sağ tarafa, kalanlar sol tarafa konur. Her kuralın güveni eşikle karşılaştırılır.
- Slayttaki algoritma:
  1. Desteği en az $c \cdot s$ olan tüm kümeleri bul.
  2. Desteği en az s olan tüm kümeleri bul.
  3. Desteği en az $c \cdot s$ olan bir $\{i_1, \ldots, i_k, j\}$ kümesi için, bir öğe çıkarıldığında kalan alt kümelerden hangilerinin desteği en az s diye bak. Bunu 2. adımın sonucundan okuyabilirsin.
  4. j'yi çıkarınca kalan kümenin desteği S_1, tüm kümenin desteği S_2 ise kuralın güveni S_2/S_1'dir.

Verinin saklanışı ve maliyet

- Pazar sepeti verisi diskte düz dosyalarda (flat file) sepet sepet tutulur.
- Sepetler arasında genelde ayıraç olarak -1 kullanılır.
- Algoritmanın maliyeti, veriyi diskten kaç kez baştan sona okuduğuna (geçiş sayısı) bağlıdır.
- Bellek yetmezse algoritma çalışamaz. Sayımın yarısını diskte yapmak pratik değildir.

Saf algoritma

- Bellek sınırı yokmuş gibi davranılır.
- Tüm olası çiftler için bir tablo kurulur. Veri bir kez okunur ve her çiftin sayısı artırılır.
- Sayısı destek eşiğinin üstünde olan çiftler sık çiftlerdir.
- Uygulanamaz, çünkü 100.000 öğe için yaklaşık 5×10^9 çift ve yaklaşık 20 GB bellek gerekir.

Çiftleri bellekte tutma

- **Üçgen matris:**
  - $i < j$ olan her çift için bir sayaç tutulur. Sayaç başına 4 bayt.
  - İki boyutlu dizi yerine tek boyutlu dizi de kullanılabilir. Bunun için çift indisini dizi indisine çeviren bir eşleme gerekir.
  - Kitaptaki eşleme ($1 \leq i < j \leq n$): $k = (i-1)\left(n - \frac{i}{2}\right) + j - i$.
- **Üçlü tablo:**
  - Her çift için $[i, j, sayı]$ tutulur: 12 bayt.
  - Karma tablosu olarak düşünülebilir. Anahtar (i, j) çiftidir, değer sayıdır.
  - Yalnız veride gerçekten görülen çiftler tutulur. Sayısı 0 olan çiftler için yer ayrılmaz.

Apriori algoritması

- Saf yaklaşımdan iyidir. Bellek ihtiyacını azaltır ama iki geçiş yapar.
- **Monotonluk özelliği:**
  - Bir öğe kümesi en az s sepette geçiyorsa tüm alt kümeleri de en az s sepette geçer.
  - Tersi: i öğesi s sepette geçmiyorsa i'yi içeren hiçbir çift s sepette geçemez.
- **Geçiş 1:**
  - Sepetler okunur, her tekil öğenin sayısı bellekte tutulur.
  - Bellek ihtiyacı öğe sayısıyla orantılıdır. 100.000 öğe için 100.000×4 bayt.
  - Sayısı s'nin altında kalanlar bellekten atılır. Yalnız sık öğeler kalır.
- **Geçiş 2:**
  - Sık öğelerden aday çiftler (candidate pairs) oluşturulur.
  - Veri yeniden okunur. Yalnız aday çiftler sayılır.
  - Sayısı s'nin altında kalanlar elenir. Kalanlar sık çiftlerdir.
- **Üçgen matris kullanılırsa:** Geçiş 1'den sonra sık olmayan öğeler atıldığı için öğeler yeniden numaralandırılır. Yeni indisi eski indise bağlayan bir eşleme tablosu tutulur. Aday çiftler yeni indislerle kurulur.

- Altyazının 01:37-01:39 arası bozuk, o kısım özete alınmadı.

PCY algoritması (Park-Chen-Yu)

- Gözlem: Apriori'nin ilk geçişinde belleğin büyük kısmı boş kalır.
- **Geçiş 1:**
  - Her sepetteki tekil öğeler sayılır.
  - Sepetteki tüm olası çiftler üretilir. Her çift bir karma fonksiyonuyla bir kovaya eşlenir ve o kovanın sayacı 1 artırılır.
- Sözde kod:

```
for (her sepet):
    for (sepetteki her öğe):
        öğenin sayacına 1 ekle
    for (sepetteki her çift):
        çifti bir kovaya karmala
        o kovanın sayacına 1 ekle
```

- **Geçişler arasında:**
  - Sık olmayan tekil öğeler atılır.
  - Karma tablosu bir bit eşleme dönüştürülür. Sayısı $\geq s$ olan kova için 1, diğerleri için 0 yazılır.
  - 32 bitlik sayaç yerine 1 bit kullanıldığı için tablo 1/32 boyuta iner.
- **Temel fikir:** Bir kova sık değilse ona karmalanan hiçbir çift sık olamaz. Kova sıksa çift sık olabilir ama kesin değildir. Bu yüzden ikinci geçişte sayılır.
- **Geçiş 2:** Bir çift ancak iki koşulu sağlarsa aday olur ve sayılır:
  - i ve j sık öğelerdir.
  - $\{i, j\}$ bit eşlemde 1 olan bir kovaya karmalanır.
- Aday sayısı Apriori'ye göre azalır, böylece bellek ve hesaplama tasarrufu sağlanır.

Multistage algoritması

- PCY'den sonra ek bir ara geçiş yapar. Toplam 3 geçiş.
- **Geçiş 1:** PCY ile aynıdır. Öğeler sayılır, çiftler 1. karma tablosuna karmalanır.
- **Geçiş 2:**
  - Bellekte sık öğeler ve 1. bit eşlem vardır.
  - Farklı bir karma fonksiyonuyla 2. karma tablosu kurulur.
  - Yalnız iki öğesi de sık olan ve 1. bit eşlemde 1'e düşen çiftler 2. tabloya karmalanır.
- **Geçiş 3:** Bir çift ancak şu koşulları sağlarsa sayılır:
  - İki öğesi de sık.
  - 1. bit eşlemde 1'e düşüyor.
  - 2. bit eşlemde 1'e düşüyor.
- Aday sayısı ve bellek ihtiyacı PCY'ye göre daha da azalır.

Multihash algoritması

- Multistage'in faydasını ek geçiş yapmadan elde etmeye çalışır. Toplam 2 geçiş.
- Geçiş 1'de boş belleğin yarısı 1. karma tablosuna, yarısı 2. karma tablosuna ayrılır. İki tablonun karma fonksiyonları farklıdır.
- Geçişler arasında iki tablo için ayrı bit eşlemler oluşturulur.
- Geçiş 2'de bir çift ancak iki öğesi de sıksa ve iki bit eşlemde de 1'e düşüyorsa sayılır.

Özet sıralama

- Saf algoritma uygulanamaz.
- Apriori, iki geçişle bellek ihtiyacını azaltır.
- PCY, birinci geçişte boş belleği kullanarak ikinci geçişteki aday sayısını azaltır.
- Multistage ve multihash, PCY'yi iki karma tablosuyla daha da iyileştirir.

Ders sonrası notlar

- **Hangi yaklaşım ne zaman:** Hoca dersin sonuna doğru "aday çiftler olası çiftlerin 1/3'ünden fazlaysa üçlü tablo mantıklı" dedi. Bu, kendi önceki açıklamasının tersidir. Doğrusu: görülen çiftler 1/3'ten azsa üçlü tablo, fazlaysa üçgen matris daha az bellek kullanır.
- **PCY ve üçgen matris:** Kitaba göre PCY'de ikinci geçişte aday çiftler düzensiz dağıldığı için üçgen matris kullanılamaz, üçlü tablo gerekir. Bu yüzden PCY, ancak olası çiftlerin en az 2/3'ünü elerse Apriori'den kazançlı çıkar.
- **Saf algoritmanın belleği:** Hoca "$n^2 \times 4$ bayt" dedi. Çiftler sırasız sayıldığı için doğrusu yaklaşık $\frac{n^2}{2} \times 4 = 2n^2$ bayttır.
- **1/32 oranı:** Hoca "her bayt 8 bit" diye açıkladı. Asıl neden şudur: 4 baytlık (32 bit) tamsayı sayaç yerine 1 bit tutulur.
- **"Varsayım" değil, özellik:** Hoca monotonluğu "algoritmanın varsayımı" diye sundu. Bu bir varsayım değil, her zaman doğru olan bir özelliktir.
- **Destek eşiği:** Hoca "3'ün üstünde" dedi. Tanım "en az s" ($\geq s$) şeklindedir. Desteği tam 3 olan küme de sıktır.
- **Altyazı sorunu:** Örnekteki son sık çift altyazıda "J and J" diye geçiyor. Kitaptaki örnekte bu çift $\{c, j\}$'dir.
- **Multihash bellek ihtiyacı:** Hoca multihash'in "naive ve Apriori'den az bellek" kullandığını söyledi. Kitaptaki asıl karşılaştırma PCY iledir. Her tablo yarı boyutta olduğu için kova sayıları büyür. Multihash, ortalama kova sayısı s'den çok küçükse fayda sağlar.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.